```python
@T.prim_func
def matmul_tir(a: T.handle, b: T.handle, c: T.handle):
  # 1. 基于循环的线程绑定
  for bx in T.thread_binding(0,16,thread="blockIdx.x"):
    for tx in T.thread_binding(0,32,thread="threadIdx.x"):
      # 2. 标量循环嵌套
      for k in range(0, 2048, 32):
        # 3. 隐式的全局内存->共享内存拷贝
        for i, j in T.grid(32, 64):
          SmemA[i, j] = A[bx*64+i, k+j]
        # 4. 标量乘法计算
        for ki, ii, jj in T.grid(32, 4, 4):
          regC[ii, jj] += smemA[..] * smemB[..]
```

(a) TVM (TensorIR)

```
#blocked = #triton_gpu.blocked<{...}>
#shared  = #triton_gpu.shared<{vec=8, perPhase=1, ...}>
#dot_op  = #triton_gpu.dot_op<{opIdx=0, parent=#blocked}>
module {
  func.func @matmul_ttgir(...) {
    // 1. 基于块级布局进行数据加载
    %a = tt.load %ptr_a : tensor<128x32xf16, #blocked>
    // 2. 隐式的共享内存数据传输
    %a_sh = triton_gpu.convert_layout %a :
tensor<128x32xf16, #blocked> -> tensor<128x32xf16, #dot_op>
    // 3. 点积运算
    %c = tt.dot %a_sh, %b_sh, %acc : tensor<128x32xf16,
#dot_op> * ...
  }
}
```

(b) Triton IR (TTGIR)

```
module {
  dp.kernel @matmul(...) {
    dp.parallel %by, %bx = (16, 16) ... {
      // 1. 显式数据分配
      %smemA = dp.alloc_buffer {scope="shared"}
      dp.for %k = 0 to 2048 step = 32 {
        // 2. 显示数据移动
        dp.copy %smemA, %A[...]
        dp.copy %smemB, %B[...]
        // 3. Block 层级语义
        dp.matmul %smemA, %smemB, %regC
      }
    }
  }
}
```

(c) DeepGen IR